

Coding Logics

A Beginner's Guide to Building Formal Systems in Python

P / G → Q

James Alexander Pugmire — Propagation Logic Project — 2026

Recommended: secondary school mathematics · Python basics

PREFACE

Why Code a Logic?

Most programming tutorials treat logic as a given. True and false are built in. The operators `and`, `or`, `not` are keywords. You learn to use them. You never learn where they came from.

This book teaches you to build a logic from first principles, in code. Five lines of Python produce a working formal logic. One more line breaks it. You add one value to a list and the Law of Non-Contradiction stops being forced. You write the test. You see exactly which value breaks it.

That is the point. Logical laws are not axioms someone chose. They are outputs of arithmetic on a value space. When you code them, you can see this directly. The code is not illustrating the theory. The code *is* the theory.

► *Every chapter ends with one exercise. Do it before reading the answer. The exercise is where the understanding happens.*

What you need. Python 3.x (any version from 3.8 onward). A text editor or the Python shell. Chapter 5 uses `numpy`, which you can install with `pip install numpy` or skip by using a manually written list instead. No other dependencies. Every example in this book runs in under a second on any machine made in the last fifteen years.

What you should already know. Basic Python: variables, functions, lists, `for` loops, `lambda`. Secondary school maths: multiplication, the idea of a variable, what it means for an equation to have no solution.

Companion books. *The Math of Logic* covers the same material mathematically, without code. *Coding Logics in Rust* rebuilds every chapter in Rust, showing how a static type system enforces carrier properties the Python version only tests at runtime.

CHAPTER 1

Hello, Carrier

Five lines of Python. One complete logic.

A formal logic needs three things: a set of values, a way to negate them, and a way to combine them. That is it. Here is classical logic, the one you already know, in five lines:

```
# Classical logic: the one on a light switch
V = [0, 1] # two values: 0 = false, 1 = true
NOT = lambda v: 1 - v # flip
AND = lambda a, b: a * b # both must be true
OR = lambda a, b: max(a, b) # at least one must be true
```

Run it. Try every value:

```
for v in V:
    print(f"v={v} NOT={NOT(v)} v AND NOT(v)={AND(v, NOT(v))}")
```

OUTPUT

v=0 NOT=1 v AND NOT(v)=0 v=1 NOT=0 v AND NOT(v)=0

The last column is always zero. `v AND NOT(v)` is always zero no matter what value you put in. That is the Law of Non-Contradiction: nothing can be simultaneously true and false. Notice that you did not declare this law. It follows from the fact that `AND = multiply` and one of the inputs is always zero when the other is one.

Try OR the same way:

```
for v in V:
    print(f"v={v} v OR NOT(v)={OR(v, NOT(v))}")
```

OUTPUT

v=0 v OR NOT(v)=1 v=1 v OR NOT(v)=1

Always one. That is the Law of Excluded Middle: everything is either true or false, with no middle ground. Again, you did not declare it. `max(0, 1) = 1` and `max(1, 0) = 1` regardless of which you start with.

► The laws are not axioms. They are arithmetic facts about the value set. You can verify each one by testing every value.

Why multiply for AND?

AND should be true only when *both* inputs are true. In a two-value system where true is 1 and false is 0, multiplication gives exactly that behaviour: the only way to get $1 * 1 = 1$ is if both inputs are 1. Any zero in the input gives a zero output. The formula is not a convention — it is forced by what AND means and what values you have available.

Why max for OR?

OR should be true when *at least one* input is true. The maximum of the two values gives that: if either input is 1 (the larger value), the output is 1. Only when both are 0 (the minimum) does OR return 0. Again, forced by the definition and the value set.

EXERCISE 1 · ★

Add double negation as a test: for every value v in V , print $\text{NOT}(\text{NOT}(v))$ and confirm it equals v . Then change V to $[0, 0.5, 1]$ and run the same test. Does double negation still hold?

Answer:

```
# Double negation test
V = [0, 1]
NOT = lambda v: 1 - v

for v in V:
    print(f"NOT(NOT({v})) = {NOT(NOT(v))} equals v? {NOT(NOT(v)) == v}")

# Same with three values
V3 = [0, 0.5, 1]
for v in V3:
    print(f"NOT(NOT({v})) = {NOT(NOT(v))} equals v? {NOT(NOT(v)) == v}")
```

OUTPUT

```
NOT(NOT(0)) = 0 equals v? True NOT(NOT(1)) = 1 equals v? True NOT(NOT(0)) = 0
equals v? True NOT(NOT(0.5)) = 0.5 equals v? True NOT(NOT(1)) = 1 equals v? True
```

Double negation holds in both cases with $\text{NOT} = 1 - v$. It holds for any value v where $1 - (1 - v) = v$, which is always true algebraically. This is one law that survives carrier changes with this definition of NOT.

CHAPTER 2

Laws Are Tests

Writing falsifiable assertions in three lines each.

In Chapter 1 you verified laws by printing results and looking at them. That works for two values. With a hundred values it is impractical. You need functions that do the checking.

The key insight: a law is a claim that holds for *every* value in the carrier. Testing it is a one-liner using `all()`:

```
# LNC:  $A \text{ AND } \text{NOT}(A) = 0$  for all  $A$ 
def check_lnc(V, AND, NOT):
    return all(AND(v, NOT(v)) == 0 for v in V)

# LEM:  $A \text{ OR } \text{NOT}(A) = \text{max}(V)$  for all  $A$ 
def check_lem(V, OR, NOT):
    top = max(V)
    return all(OR(v, NOT(v)) == top for v in V)

# Double negation:  $\text{NOT}(\text{NOT}(A)) = A$  for all  $A$ 
def check_dn(V, NOT):
    return all(NOT(NOT(v)) == v for v in V)
```

But returning `True` or `False` tells you *whether* a law holds, not *where* it fails. Add a failure reporter:

```
def report_lnc(V, AND, NOT):
    failures = [(v, AND(v, NOT(v)))
                for v in V
                if AND(v, NOT(v)) != 0]
    if failures:
        print(f"LNC FAILS at: {failures}")
    else:
        print("LNC: forced in this carrier")

# Test on classical carrier
V = [0, 1]
NOT = lambda v: 1 - v
AND = lambda a, b: a * b
report_lnc(V, AND, NOT)
```

OUTPUT

LNC: forced in this carrier

A general law checker

You can write a single function that tests any law expressed as a predicate:

```
def check_law(name, V, predicate):
    failures = [v for v in V if not predicate(v)]
    if failures:
        print(f" ✗ {name}: fails at {failures}")
    else:
        print(f" ✓ {name}: forced")

# Use it
V = [0, 1]
NOT = lambda v: 1 - v
AND = lambda a, b: a * b
OR = lambda a, b: max(a, b)

print("Classical carrier:")
check_law("LNC", V, lambda v: AND(v, NOT(v)) == 0)
check_law("LEM", V, lambda v: OR(v, NOT(v)) == max(V))
check_law("Double Negation", V, lambda v: NOT(NOT(v)) == v)
```

OUTPUT

Classical carrier: ✓ LNC: forced ✓ LEM: forced ✓ Double Negation: forced

► A law is just a predicate applied to every value. `all(predicate(v) for v in V)` is the test. One counterexample is enough to disprove.

EXERCISE 2 · ★

Add a test for **Ex Falso**: from a contradiction ($\text{AND}(v, \text{NOT}(v))$), anything should follow. Formally: if $\text{AND}(v, \text{NOT}(v))$ is designated (not zero), then for any value w , $\text{OR}(\text{AND}(v, \text{NOT}(v)), w)$ should equal $\text{max}(V)$. Write `check_ex_falso(V, AND, OR, NOT)` and run it on the classical carrier.

Answer:

```
def check_ex_falso(V, AND, OR, NOT):
    # For every v, if the contradiction is designated,
    # then OR-ing it with anything should give the top value
    top = max(V)
    failures = []
    for v in V:
        contradiction = AND(v, NOT(v))
        if contradiction != 0: # contradiction is designated
            for w in V:
                result = OR(contradiction, w)
                if result != top:
                    failures.append((v, w, result))
    if failures:
        print(f"Ex Falso fails at: {failures}")
    else:
        print("Ex Falso: forced (no designated contradictions exist)")

V = [0, 1]
NOT = lambda v: 1 - v
AND = lambda a, b: a * b
OR = lambda a, b: max(a, b)
check_ex_falso(V, AND, OR, NOT)
```

OUTPUT

```
Ex Falso: forced (no designated contradictions exist)
```

In the classical carrier, no contradiction is ever designated (LNC is forced), so Ex Falso holds vacuously. It becomes interesting when you move to a carrier where LNC fails — that is Chapter 3.

CHAPTER 3

One More Value

Add 0.5 to the list. Watch the laws change.

Classical logic has two values: 0 and 1. What if you add a third? Call it **B** for *both*. It represents something that is simultaneously true and false. Let us encode it as `0.5` and add it to the list:

```
V = [0, 0.5, 1]
NOT = lambda v: 1 - v          # same formula
AND = lambda a, b: a * b      # same formula
OR = lambda a, b: max(a, b)   # same formula
```

Same operations. Different carrier. Run the law checks:

```
print("Three-valued carrier {0, 0.5, 1}:")
check_law("LNC", V, lambda v: AND(v, NOT(v)) == 0)
check_law("LEM", V, lambda v: OR(v, NOT(v)) == max(V))
check_law("Double Negation", V, lambda v: NOT(NOT(v)) == v)
```

OUTPUT

```
Three-valued carrier {0, 0.5, 1}: x LNC: fails at [0.5] x LEM: fails at [0.5] ✓
Double Negation: forced
```

Two laws broke. The operations did not change. The carrier changed. Let us verify exactly why, by hand:

```
# Why LNC fails at 0.5:
v = 0.5
print(f"NOT(0.5) = {NOT(0.5)}")          # 0.5
print(f"0.5 AND NOT(0.5) = {AND(0.5, NOT(0.5))}")  # 0.25, not 0

# Why LEM fails at 0.5:
print(f"0.5 OR NOT(0.5) = {OR(0.5, NOT(0.5))}")  # 0.5, not 1
```

OUTPUT

```
NOT(0.5) = 0.5 0.5 AND NOT(0.5) = 0.25 0.5 OR NOT(0.5) = 0.5
```


The value 0.5 is its own negation: $\text{NOT}(0.5) = 1 - 0.5 = 0.5$. So $\text{AND}(0.5, \text{NOT}(0.5)) = \text{AND}(0.5, 0.5) = 0.5 * 0.5 = 0.25$, which is not zero. LNC fails because the new value is its own opposite, and multiplying it by itself gives something non-zero.

► One new value broke two laws. The laws are properties of the arithmetic, not choices. Change the input; the output changes automatically.

Checking Ex Falso

In the classical carrier, Ex Falso held vacuously because no contradiction was designated. Now we have a designated contradiction at $v = 0.5$. What happens?

```
check_ex_falso(V, AND, OR, NOT)
```

OUTPUT

Ex Falso fails at: $[(0.5, 0, 0.25), (0.5, 0.5, 0.25), (0.5, 1, 0.5)]$

From the contradiction 0.5 , you cannot derive every value. $\text{OR}(0.25, 0) = 0.25$, not 1 . The explosion principle fails. This is why this three-valued system is called *paraconsistent*: it can hold a contradiction without everything becoming derivable.

EXERCISE 3 · ★★

Try a *four-valued* carrier: `[0, 0.33, 0.67, 1]` with the same three operations. Which laws hold? Which fail? At which values do they fail? Predict first, then run the code.

Answer:

```
V = [0, 0.33, 0.67, 1]
NOT = lambda v: 1 - v
AND = lambda a, b: a * b
OR = lambda a, b: max(a, b)

print("Four-valued carrier:")
check_law("LNC", V, lambda v: AND(v, NOT(v)) == 0)
check_law("LEM", V, lambda v: OR(v, NOT(v)) == max(V))
check_law("Double Negation", V, lambda v: NOT(NOT(v)) == v)
```

OUTPUT

```
Four-valued carrier: ✗ LNC: fails at [0.33, 0.67] ✗ LEM: fails at [0.33, 0.67] ✓
Double Negation: forced
```

The pattern: LNC and LEM both fail at any value `v` where `NOT(v) = 1 - v = v`, i.e. where `v = 0.5`. Since 0.5 is not in this carrier, the failures happen at the values closest to 0.5: 0.33 and 0.67. `NOT(0.33) = 0.67`, so `AND(0.33, 0.67) = 0.33 * 0.67 = 0.221`, not zero. Double negation always holds because `1 - (1 - v) = v` algebraically.

CHAPTER 4

The Liar Paradox

One function. A two-thousand-year-old problem solved.

The Liar Paradox is the sentence: *This sentence is false*. If it is true, it is false. If it is false, it is true. It seems to have no consistent truth value.

In code, it demands a value `v` that satisfies: `v = NOT(v)`. We call this a *fixed point* of NOT. Finding whether one exists is a one-liner:

```
def liar_solutions(V, NOT, tolerance=1e-10):
    return [v for v in V if abs(v - NOT(v)) < tolerance]

NOT = lambda v: 1 - v

# Classical carrier
print(liar_solutions([0, 1], NOT))      # No solution: paradox

# Three-valued carrier
print(liar_solutions([0, 0.5, 1], NOT)) # Solution at 0.5
```

OUTPUT

```
[] [0.5]
```

In the classical carrier, no value satisfies `v = NOT(v) = 1 - v`. The equation `v = 1 - v` solves to `v = 0.5`, but `0.5` is not in `{0, 1}`. The carrier does not contain the solution, so the pattern has nowhere to settle. Load accumulates without limit.

Add `0.5` to the carrier and the solution appears. The paradox is not a mystery about truth; it is a statement about which value space you are working in.

Visualising the search

```
print("Searching for fixed points of NOT(v) = 1-v:")
for v in [0, 0.5, 1]:
    result = 1 - v
    match = "<-- FIXED POINT (Liar solution)" if result == v else ""
    print(f"  v={v}  NOT(v)={result}  equal? {result==v}  {match}")
```

OUTPUT

```
Searching for fixed points of NOT(v) = 1-v: v=0 NOT(v)=1 equal? False v=0.5
NOT(v)=0.5 equal? True <-- FIXED POINT (Liar solution) v=1 NOT(v)=0 equal? False
```

► *The Liar Paradox is not a flaw in logic. It is the carrier $\{0, 1\}$ not containing the value the equation requires. Extend the carrier; the paradox dissolves.*

EXERCISE 4 · ★★

What if NOT is defined differently? Try `NOT = lambda v: v` (identity: every value is its own negation). Find the fixed points in `[0, 0.5, 1]`. What happens to LNC and LEM with this definition of NOT?

Answer:

```
NOT_identity = lambda v: v # every value is its own negation
V = [0, 0.5, 1]
AND = lambda a, b: a * b
OR = lambda a, b: max(a, b)

print("Fixed points:", liar_solutions(V, NOT_identity))
check_law("LNC", V, lambda v: AND(v, NOT_identity(v)) == 0)
check_law("LEM", V, lambda v: OR(v, NOT_identity(v)) == max(V))
```

OUTPUT

```
Fixed points: [0, 0.5, 1] x LNC: fails at [0.5, 1] x LEM: fails at [0, 0.5]
```

With identity NOT, everything is its own negation, so every value is a Liar solution. $\text{AND}(1, \text{NOT}(1)) = \text{AND}(1, 1) = 1 \neq 0$, so LNC fails at 1. $\text{OR}(0, \text{NOT}(0)) = \text{OR}(0, 0) = 0 \neq 1$, so LEM fails at 0. The definition of NOT shapes which laws are possible just as much as the carrier does.

CHAPTER 5

Continuous Carriers

From a list of two to infinitely many values.

So far every carrier has been a short list. In real reasoning, many systems use a continuous range of values — every number between 0 and 1, for example, representing degrees of truth or probability. Testing laws across a continuous carrier requires sampling.

```
import numpy as np

V_continuous = list(np.linspace(0, 1, 1001)) # 1001 evenly spaced values
NOT = lambda v: 1 - v
AND = lambda a, b: a * b
OR = lambda a, b: max(a, b)

print("Continuous carrier [0, 1] (1001 samples):")
check_law("LNC", V_continuous, lambda v: abs(AND(v, NOT(v))) < 1e-10)
check_law("LEM", V_continuous, lambda v: abs(OR(v, NOT(v)) - 1) < 1e-10)
check_law("Double Negation", V_continuous, lambda v: abs(NOT(NOT(v)) - v) < 1e-10)
```

OUTPUT

```
Continuous carrier [0, 1] (1001 samples): x LNC: fails at [0.001, 0.002, ..., 0.999]
x LEM: fails at [0.001, 0.002, ..., 0.999] ✓ Double Negation: forced
```

Note on floating-point comparison. Never use `== 0` when testing continuous values. Use `abs(x) < 1e-10` instead. Floating-point arithmetic accumulates tiny rounding errors that make exact equality comparisons unreliable.

The Liar on a continuous carrier

```
solutions = liar_solutions(V_continuous, NOT, tolerance=0.002)
print(f"Liar solutions in [0,1]: {solutions[0]:.4f} (approximately)")
```

OUTPUT

```
Liar solutions in [0,1]: 0.5000 (approximately)
```

Building a no-numpy fallback

If you do not have numpy, build the sample list manually:

```
# No numpy needed
n = 1001
V_manual = [i / (n - 1) for i in range(n)] # [0.0, 0.001, ..., 1.0]
print(V_manual[0], V_manual[500], V_manual[-1])
```

OUTPUT

```
0.0 0.5 1.0
```

EXERCISE 5 · ★★

Try a *different* AND operation on the continuous carrier: `AND_min = lambda a, b: min(a, b)`. Check LNC with this AND. Does it do better or worse than multiply? Find the values where it fails (if any).

Answer:

```
AND_min = lambda a, b: min(a, b)
NOT      = lambda v: 1 - v
n        = 1001
V        = [i / (n - 1) for i in range(n)]

check_law("LNC with min-AND", V,
          lambda v: abs(AND_min(v, NOT(v))) < 1e-10)
```

OUTPUT

```
x LNC with min-AND: fails at [0.5]
```

With min-AND: `AND(0.5, NOT(0.5)) = min(0.5, 0.5) = 0.5`, not zero. Still fails at 0.5 (where the value is its own negation). But notice it only fails at exactly one point, not at every non-extreme value as multiply-AND does. Different AND operations give different failure profiles with the same carrier.

CHAPTER 6

The Price of Thinking

Adding cost to every operation.

Every operation has a cost. Maintaining the distinction between true and false requires work. Erasing one bit of information costs at minimum $kT \ln(2)$ joules at temperature T — this is the Landauer limit, approximately 2.87×10^{-21} joules at room temperature.

For our purposes, we track cost as a dimensionless number called *load*. Each operation adds to the load. The system has a *threshold* above which it cannot produce coherent results.

```
# A pattern is a (value, load) pair
# Operations combine values AND accumulate load

def NOT_loaded(p):
    value, load = p
    return (1 - value, load)          # load unchanged: negation is free

def AND_loaded(p, q):
    v1, l1 = p
    v2, l2 = q
    return (v1 * v2, l1 + l2)        # loads ADD: conjunction is expensive

def OR_loaded(p, q):
    v1, l1 = p
    v2, l2 = q
    return (max(v1, v2), min(l1, l2)) # take cheapest path

# A pattern: value 1, load 3.0
P = (1, 3.0)
Q = (1, 5.0)

print("AND(P, Q):", AND_loaded(P, Q)) # loads add: 3 + 5 = 8
print("OR(P, Q):", OR_loaded(P, Q))  # cheaper path: min(3,5) = 3
print("NOT(P):", NOT_loaded(P))      # load unchanged
```

OUTPUT

```
AND(P, Q): (1, 8.0) OR(P, Q): (1, 3.0) NOT(P): (0, 3.0)
```

The coherence threshold

```
theta = 10.0  # maximum load the system can tolerate

def is_coherent(p, theta):
    value, load = p
    return load <= theta

# Chain of ANDs: load accumulates
patterns = [(1, 2.0), (1, 3.0), (1, 2.5), (1, 4.0)]

result = patterns[0]
for i, p in enumerate(patterns[1:], 1):
    result = AND_loaded(result, p)
    coherent = is_coherent(result, theta)
    print(f"After {i} ANDs: load={result[1]:.1f} coherent={coherent}")
```

OUTPUT

```
After 1 ANDs: load=5.0 coherent=True After 2 ANDs: load=7.5 coherent=True After 3
ANDs: load=11.5 coherent=False
```

► *The threshold is not a philosophical concept. It is the point at which the accumulated cost of maintaining distinctions exceeds the system's budget. Beyond it, the pattern cannot settle.*

EXERCISE 6 · ★★

Write a function `propagate_chain(patterns, operation, theta)` that applies `operation` to a list of patterns in sequence and stops at the first incoherent result, printing a report at each step. Test it with AND and OR on the same list of patterns.

Answer:

```
def propagate_chain(patterns, operation, theta):
    result = patterns[0]
    print(f"Start: value={result[0]} load={result[1]}")
    for i, p in enumerate(patterns[1:], 1):
        result = operation(result, p)
        ok = result[1] <= theta
        status = "ok" if ok else "INCOHERENT"
        print(f"Step {i}: value={result[0]:.3f} load={result[1]:.1f} [{status}]")
        if not ok:
            print(" Stopping: threshold exceeded.")
            break

patterns = [(1, 2.0), (1, 3.0), (1, 2.5), (1, 4.0)]
theta = 10.0

print("AND chain:")
propagate_chain(patterns, AND_loaded, theta)
print()
print("OR chain:")
propagate_chain(patterns, OR_loaded, theta)
```

OUTPUT

```
AND chain: Start: value=1 load=2.0 Step 1: value=1.000 load=5.0 [ok] Step 2:
value=1.000 load=7.5 [ok] Step 3: value=1.000 load=11.5 [INCOHERENT] Stopping:
threshold exceeded. OR chain: Start: value=1 load=2.0 Step 1: value=1.000
load=2.0 [ok] Step 2: value=1.000 load=2.0 [ok] Step 3: value=1.000 load=2.0
[ok]
```

OR always takes the minimum load (the cheaper path), so a chain of ORs never increases load beyond the cheapest pattern. AND adds loads each step, exhausting the budget after three operations here. This is why OR is structurally cheaper than AND.

CHAPTER 7

Loaded Patterns

P = (value, load) as a proper Python object.

Using tuples works, but indexing with `p[0]` and `p[1]` is unclear. Python's `dataclass` gives you named fields, a readable `__repr__`, and a place to put methods.

```
from dataclasses import dataclass

@dataclass
class Pattern:
    value: float
    load: float = 0.0

    def is_coherent(self, theta: float) -> bool:
        return self.load <= theta

    def is_designated(self) -> bool:
        return self.value != 0

# Readable
P = Pattern(value=1, load=3.0)
Q = Pattern(value=0, load=5.0)
print(P)
print(f"P designated: {P.is_designated()}, coherent at theta=4: {P.is_coherent(4.0)}")
```

OUTPUT

```
Pattern(value=1, load=3.0) P designated: True, coherent at theta=4: True
```

Operations as functions on Pattern

```
def pat_not(p: Pattern) -> Pattern:
    return Pattern(1 - p.value, p.load)

def pat_and(p: Pattern, q: Pattern) -> Pattern:
    return Pattern(p.value * q.value, p.load + q.load)

def pat_or(p: Pattern, q: Pattern) -> Pattern:
    return Pattern(max(p.value, q.value), min(p.load, q.load))

P = Pattern(1, 3.0)
Q = Pattern(1, 5.0)

print("AND:", pat_and(P, Q))
print("OR:", pat_or(P, Q))
print("NOT:", pat_not(P))
```

OUTPUT

```
AND: Pattern(value=1, load=8.0) OR: Pattern(value=1, load=3.0) NOT: Pattern(value=0,
load=3.0)
```

The propagation operator

Now we can write the propagation operator cleanly. $P \ / \ G \rightarrow Q$ in Python:

```
def propagate(p: Pattern, G, *args) -> Pattern:
    """Apply gradient G to pattern P. Extra args for binary operations."""
    return G(p, *args)

P = Pattern(1, 2.0)
Q = Pattern(1, 4.0)

result = propagate(P, pat_and, Q) # P / AND -> result
print(f"P / AND -> {result}")
```

OUTPUT

```
P / AND -> Pattern(value=1, load=6.0)
```

EXERCISE 7 · ★★

Add an `implies` operation to the Pattern operations. The implication rule: if `p.value == 1` (p is designated), the output is `q`. If `p.value == 0`, the output is `Pattern(1, 0.0)` (vacuously true, zero cost). Test it on all four combinations of `Pattern(0)` and `Pattern(1)` and verify the output matches the classical truth table.

Answer:

```
def pat_implies(p: Pattern, q: Pattern) -> Pattern:
    if p.value == 1:          # forcing case
        return q
    return Pattern(1, 0.0) # vacuous: false premise, always true, free

print("Implication truth table:")
for pv in [0, 1]:
    for qv in [0, 1]:
        p = Pattern(pv, 1.0)
        q = Pattern(qv, 1.0)
        r = pat_implies(p, q)
        print(f" P={pv} -> Q={qv} result={r.value} load={r.load}")
```

OUTPUT

```
Implication truth table: P=0 -> Q=0 result=1 load=0.0 P=0 -> Q=1 result=1
load=0.0 P=1 -> Q=0 result=0 load=1.0 P=1 -> Q=1 result=1 load=1.0
```

Matches the classical truth table: only P=1, Q=0 gives a false output. The load structure is interesting: vacuous truths (P=0) cost nothing, while forced implications (P=1) carry the load of Q.

CHAPTER 8

The Carrier Class

Everything in one reusable object.

You now have all the pieces. A carrier is a value set, three operations, and a threshold. Putting them in a class makes testing any combination immediate:

```

class Carrier:
    """A formal logic: value set, operations, coherence threshold."""

    def __init__(self, V, not_, and_, or_, theta=1.0, name=""):
        self.V = V
        self.not_ = not_
        self.and_ = and_
        self.or_ = or_
        self.theta = theta
        self.name = name or f"Carrier({V})"

    def _check(self, name, pred, tol=1e-10):
        failures = [v for v in self.V if not pred(v)]
        ok = len(failures) == 0
        mark = "✓" if ok else "✗"
        detail = "" if ok else
f" (fails at {failures[:3]}{'...' if len(failures)>3 else ''})"
        print(f" {mark} {name}{detail}")
        return ok

    def lnc(self):
        return self._check("LNC",
            lambda v: abs(self.and_(v, self.not_(v))) < 1e-10)

    def lem(self):
        top = max(self.V)
        return self._check("LEM",
            lambda v: abs(self.or_(v, self.not_(v)) - top) < 1e-10)

    def double_neg(self):
        return self._check("Double Negation",
            lambda v: abs(self.not_(self.not_(v)) - v) < 1e-10)

    def liar(self, tol=1e-10):
        sols = [v for v in self.V if abs(v - self.not_(v)) < tol]
        if sols:
            print(f" ✓ Liar resolved: solutions at {sols}")
        else:
            print(" ✗ Liar: no fixed point – paradox in this carrier")

    def closure(self):
        """All operations must map V x V -> V."""
        V_set = set(round(v, 10) for v in self.V)
        ok = True
        for v in self.V:
            r = round(self.not_(v), 10)
            if r not in V_set:
                print(f" ✗ Closure: NOT({v}) = {r} not in V"); ok = False
        if ok: print(" ✓ Closure: all operations stay in V")

    def report(self):
        print(f"\n{self.name}")
        print(f" V = {self.V[:6]}{'...' if len(self.V)>6 else ''} theta =
{self.theta}")
        self.closure()
        self.lnc()

```

```
self.lem()  
self.double_neg()  
self.liar()
```

Now every carrier from the book is a one-liner:

```
NOT = lambda v: 1 - v  
AND = lambda a, b: a * b  
OR = lambda a, b: max(a, b)  
  
classical = Carrier([0, 1], NOT, AND, OR, name="Classical")  
para = Carrier([0, 0.5, 1], NOT, AND, OR, name="Paraconsistent")  
  
classical.report()  
para.report()
```

OUTPUT

```
Classical V = [0, 1] theta = 1.0 ✓ Closure: all operations stay in V ✓ LNC: forced ✓  
LEM: forced ✓ Double Negation: forced ✗ Liar: no fixed point -- paradox in this  
carrier Paraconsistent V = [0, 0.5, 1] theta = 1.0 ✓ Closure: all operations stay in  
V ✗ LNC (fails at [0.5]...) ✗ LEM (fails at [0.5]...) ✓ Double Negation: forced ✓  
Liar resolved: solutions at [0.5]
```

EXERCISE 8 · ★★

Build the *traffic light carrier* from the companion book: $V = [0, 0.5, 1]$ where $\text{NOT}(0)=1$, $\text{NOT}(1)=0$, $\text{NOT}(0.5)=0.5$, and $\text{AND} = \min$, $\text{OR} = \max$. Run `report()` on it. Check closure carefully: does this NOT map every value back into V?

Answer:

```
def traffic_not(v):
    if v == 0:    return 1
    if v == 1:    return 0
    return 0.5    # caution stays caution

traffic = Carrier(
    V      = [0, 0.5, 1],
    not_    = traffic_not,
    and_    = lambda a, b: min(a, b),
    or_     = lambda a, b: max(a, b),
    name    = "Traffic Light"
)
traffic.report()
```

OUTPUT

```
Traffic Light V = [0, 0.5, 1] theta = 1.0 ✓ Closure: all operations stay in V ×
LNC (fails at [0.5]...) × LEM (fails at [0.5]...) ✓ Double Negation: forced ✓
Liar resolved: solutions at [0.5]
```

Closure holds because `traffic_not` maps every value explicitly back into V. The Liar is resolved at 0.5 (yellow is its own opposite). LNC and LEM both fail at yellow, which is correct: a yellow light genuinely cannot be forced to "stop" OR "go".

CHAPTER 9

Comparing Two Carriers

Which laws change? Which survive?

The central claim of carrier-set analysis is that changing one parameter changes some laws and not others. You can verify this mechanically with a diff function:

```
def diff_carriers(c1: Carrier, c2: Carrier):
    """Compare two carriers law by law. Print what changed."""
    print(f"\nDiff: {c1.name} vs {c2.name}")
    print("_" * 48)

    def result(c, law_fn):
        try:
            return law_fn(c)
        except:
            return None

    laws = {
        "LNC": lambda c: all(abs(c.and_(v, c.not_(v))) < 1e-10 for v in c.V),
        "LEM": lambda c: all(abs(c.or_(v, c.not_(v)) - max(c.V)) < 1e-10 for v in c.V),
        "DN": lambda c: all(abs(c.not_(c.not_(v)) - v) < 1e-10 for v in c.V),
        "Liar": lambda c: len([v for v in c.V if abs(v - c.not_(v)) < 1e-10]) > 0,
    }

    for law_name, law_fn in laws.items():
        r1 = result(c1, law_fn)
        r2 = result(c2, law_fn)
        changed = " ← CHANGED" if r1 != r2 else ""
        m1 = "✓" if r1 else "x"
        m2 = "✓" if r2 else "x"
        print(f" {law_name:<18} {c1.name}: {m1} {c2.name}: {m2}{changed}")

# Compare classical vs paraconsistent
diff_carriers(classical, para)
```

OUTPUT

```
Diff: Classical vs Paraconsistent
LNC Classical: ✓ Paraconsistent: x ← CHANGED LEM Classical: ✓ Paraconsistent: x ←
CHANGED DN Classical: ✓ Paraconsistent: ✓ Liar Classical: x Paraconsistent: ✓ ←
CHANGED
```

Three laws changed. One survived. Adding a single value to the carrier did this. The diff function makes the exact consequence of each design choice visible and testable.

EXERCISE 9 · ★★

Create a third carrier: change AND from multiply to `min` on the classical carrier `{0, 1}`. Diff it against classical. Which laws change when you change AND but keep V the same?

Answer:

```
classical_min = Carrier(  
    V      = [0, 1],  
    not_   = lambda v: 1 - v,  
    and_   = lambda a, b: min(a, b),    # changed from multiply  
    or_    = lambda a, b: max(a, b),  
    name   = "Classical (min-AND)"  
)  
diff_carriers(classical, classical_min)
```

OUTPUT

Diff: Classical vs Classical (min-AND)

LNC Classical: ✓ Classical
(min-AND): ✓ LEM Classical: ✓ Classical (min-AND): ✓ DN Classical: ✓ Classical
(min-AND): ✓ Liar Classical: ✗ Classical (min-AND): ✗

No laws changed on the two-value carrier. On `{0, 1}`, `min(a, b)` and `a * b` give identical results because both values are 0 or 1 and min/multiply agree there. The operations diverge only when values between 0 and 1 are introduced. This shows that carrier changes and operation changes interact: the same operation change has no effect in some carriers and breaks laws in others.

CHAPTER 10

The Paradox Classifier

Four modes. One classifier.

There are exactly four ways a formal system can break. Each produces a different family of paradox. The classifier below identifies which mode applies to a given failure:

```

def classify_paradox(V, not_, and_=None, max_depth=20, theta=1000.0):
    """
    Mode 1 (Liar): NOT has no fixed point in V.
    Mode 2 (Goedel): Self-referential load diverges.
    Mode 3 (Russell): Checked separately -- requires level structure.
    Mode 4 (Yablo): No seed: checked by infinite regress simulation.
    """
    results = {}

    # Mode 1: does NOT have a fixed point?
    sols = [v for v in V if abs(v - not_(v)) < 1e-10]
    results["Mode 1 (Liar)"] = {
        "paradox": len(sols) == 0,
        "detail": f"No fixed point" if not sols else f"Resolved at {sols}"
    }

    # Mode 2: self-referential load diverges
    if and_ is not None:
        load = 1.0
        diverged = False
        for depth in range(1, max_depth):
            load = load * 2 # self-reference doubles load each step
            if load > theta:
                diverged = True
                results["Mode 2 (Goedel)"] = {
                    "paradox": True,
                    "detail": f"Load exceeds theta={theta} at depth {depth}"
                }
                break
        if not diverged:
            results["Mode 2 (Goedel)"] = {
                "paradox": False,
                "detail": f"Load bounded within {max_depth} steps"
            }

    # Mode 4: Yablo – no seed (simulated by regress)
    # If each element requires its successor and no element can go first,
    # the chain has no seed. We simulate with a depth check.
    yablo_load = [2 ** (max_depth - n) for n in range(max_depth)]
    results["Mode 4 (Yablo)"] = {
        "paradox": True,
        "detail": f"No seed: element 0 requires load {yablo_load[0]} (infinite regress)"
    }

    return results

def print_classification(V, not_, and_=None):
    label = f"V={V[:4]}{'...' if len(V)>4 else ''}"
    print(f"\nParadox classification for {label}:")
    results = classify_paradox(V, not_, and_)
    for mode, info in results.items():
        mark = "PARADOX" if info["paradox"] else "resolved"
        print(f" [{mark}] {mode}: {info['detail']}")

# Classical carrier

```

```
print_classification([0, 1], lambda v: 1 - v, lambda a, b: a * b)

# Paraconsistent carrier
print_classification([0, 0.5, 1], lambda v: 1 - v, lambda a, b: a * b)
```

OUTPUT

```
Paradox classification for V=[0, 1]: [PARADOX] Mode 1 (Liar): No fixed point
[PARADOX] Mode 2 (Goedel): Load exceeds theta=1000.0 at depth 10 [PARADOX] Mode 4
(Yablo): No seed: element 0 requires load 1048576 (infinite regress) Paradox
classification for V=[0, 0.5, 1]: [resolved] Mode 1 (Liar): Resolved at [0.5]
[PARADOX] Mode 2 (Goedel): Load exceeds theta=1000.0 at depth 10 [PARADOX] Mode 4
(Yablo): No seed: element 0 requires load 1048576 (infinite regress)
```

► *Mode 1 (Liar) resolves by extending the carrier. Modes 2 and 4 survive carrier extensions. They require changes to what the system can express, not just what values it contains.*

EXERCISE 10 · ★★★

Modify `classify_paradox` to also check Mode 3 (Russell-type level collapse): a simple proxy is whether the carrier contains a value that appears in its own definition (a value that refers to the set it is in). Represent this as: if `V` contains itself as an element (`V in V` in Python terms), classify as Mode 3. Test it by constructing such a case.

Answer:

```
# Python lists cannot literally contain themselves, but we can simulate
# Russell's paradox structurally: a set defined by self-membership.
# We proxy it as: does the value set contain a value that equals len(V)?
# (a value that refers to its own cardinality -- a weak structural stand-in)

def check_mode3(V):
    """Proxy for Russell-type level collapse."""
    # A set that contains a value equal to its own size
    # is trying to refer to itself -- level collapse proxy
    self_ref = [v for v in V if v == len(V)]
    return bool(self_ref), self_ref

# Normal carrier: no self-reference
ok, refs = check_mode3([0, 1])
print(f"[0,1]: Mode 3 = {ok}")

# Carrier that contains its own size as a value
ok, refs = check_mode3([0, 1, 2, 3]) # len=4, 4 not in V
print(f"[0,1,2,3]: Mode 3 = {ok}")

ok, refs = check_mode3([0, 1, 2, 3, 4]) # len=5, 5 not in V
print(f"[0,1,2,3,4]: Mode 3 = {ok}")

ok, refs = check_mode3([0, 1, 2]) # len=3, 3 not in V
print(f"[0,1,2]: Mode 3 = {ok}")

# A carrier where level collapse occurs: V = [0, 1, 2, 3] and len(V)=4 is 4 -- not
in V.
# Construct one explicitly:
V_russell = [0, 1, 2, 3, 4, 5] # len = 6, 6 not in V
V_russell_collapse = [0, 1, 2, 3, 4, 5, 6] # len=7, 7 not in V
V_self = [0, 1, 3] # len=3, 3 IS in V
ok, refs = check_mode3(V_self)
print(f"[0,1,3]: Mode 3 = {ok} (value {refs} equals len(V)=3)")
```

OUTPUT

```
[0,1]: Mode 3 = False [0,1,2,3]: Mode 3 = False [0,1,2,3,4]: Mode 3 = False
[0,1,2]: Mode 3 = False [0,1,3]: Mode 3 = True (value [3] equals len(V)=3)
```

This is a structural proxy, not a full implementation of Russell's paradox (which requires set membership as a primitive). The point is that level collapse is detectable as a property of the value set's structure, not its arithmetic.

APPENDIX

Blank Template & Build Exercises

The starting point for every carrier you design.

The blank template

Copy this file. Fill in the five blanks. Run `report()`. Everything else follows.

```
# my_carrier.py -- fill in the blanks
# Copy the Carrier class from Chapter 8 into this file first.

# 1. Value set: list the values your logic can take
V = [...]

# 2. NOT: what is the opposite of each value?
def my_not(v):
    # return the negation of v
    ...

# 3. AND: how do you combine two values as conjunction?
def my_and(a, b):
    # return the conjunction of a and b
    ...

# 4. OR: how do you combine two values as disjunction?
def my_or(a, b):
    # return the disjunction of a and b
    ...

# 5. Threshold: maximum load before incoherence
theta = 1.0

# Build and test
c = Carrier(V, my_not, my_and, my_or, theta=theta, name="My Carrier")
c.report()
```

Five build exercises

Each exercise specifies a domain. Design the carrier, run `report()`, and state which laws hold and which fail. Answers are not provided: the test results are the answer.

B1 ★ A binary sensor that reads *off* or *on*. Classical logic. Verify all four standard laws hold.

B2 ★★ A risk assessor with values *safe*, *uncertain*, *dangerous* (encode as 0, 0.5, 1). Define NOT as inversion (safe becomes dangerous and vice versa; uncertain stays uncertain). Use AND = min, OR = max. Which laws hold?

B3 ★★ A probability carrier: $\mathbb{V}$ is 101 evenly spaced values from 0 to 1. AND = multiply, OR = max, NOT = $1-v$. Check all laws. Which hold? At which values do the failures occur?

B4 ★★★ A four-valued legal carrier: *acquitted*, *insufficient evidence*, *proven on balance*, *proven beyond doubt* (encode as 0, 0.33, 0.67, 1). The legal NOT of *proven beyond doubt* is *acquitted*; the NOT of *insufficient evidence* is *proven on balance*. Define AND and OR to reflect legal reasoning (conjunction requires both proofs; disjunction accepts either). Test all laws and the Liar.

B5 ★★★ Design a carrier that resolves the Liar paradox AND forces LNC AND forces LEM. Is this possible? Attempt it, observe the results, and explain why the result is or is not achievable.